

Practical-4

Aim

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Files

- practical_04.c
- README.md
- output.txt

Compilation

```
gcc practical_4.c -o practical_4
```

Execution

```
./practical_04
```

Result

The program successfully demonstrates process synchronization using both wait() and waitpid().

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    pid_t pid1, pid2;
```

```
pid1 = fork();
```

```
if (pid1 == 0)
```

```
{
```

```
    printf("Child 1 Started\n");
```

```
    printf("PID : %d\n", getpid());
```

```
    printf("PPID : %d\n", getppid());
```

```
    sleep(3);
```

```
    printf("Child 1 Finished\n");
```

```
    exit(1);
```

```
}
```

```
pid2 = fork();
```

```
if (pid2 == 0)
```

```
{
```

```
    printf("\nChild 2 Started\n");
```

```
    printf("PID : %d\n", getpid());
```

```
    printf("PPID : %d\n", getppid());
```

```
    sleep(5);
```

```
    printf("Child 2 Finished\n");
```

```
    exit(2);
```

```
}
```

```
int status;

printf("\nParent Waiting using wait()...\n");

wait(&status);

printf("One child has terminated.\n");

printf("\nParent Waiting for Child 2 using waitpid()...\n");

waitpid(pid2, &status, 0);

printf("Child 2 has terminated.\n");

printf("\nParent Process Finished.\n");

return 0;
}
```

Output -

Child 1 Started

PID : 1159

PPID : 1158

Child 1 Finished

Child 2 Started

PID : 1160

PPID : 1158

Child 2 Finished

Parent Waiting using wait()...

One child has terminated.

Parent Waiting for Child 2 using waitpid()...

Child 2 has terminated.

Parent Process Finished.